Содержание

4 Задача о рюкзаке **10**
4.1 Линейная релаксация . . . 10
4.2 Верхние и нижние границы . . . 11
4.3 Жадный алгоритм . . . 11
4.4 Аппроксимация с помощью полиномиального алгоритма . . . 12
4.5 Решение динамическим программированием . . . 13
4.6 Гибридный алгоритм: жадный и динамическое программирование . . . 14
4.7 Алгоритм ветвей и границ . . . 15
4.8 Фиксация переменных и упрощение задачи . . . 16
4.9 Ядро и упрощённая задача . . . 17
4.10 Задача о сумме подмножеств . . . 17

4 Задача о рюкзаке

Название проблемы происходит от следующего её представления. Пусть имеются n объектов с весом w_i и ценой v_i. Необходимо собрать некоторые из объектов в рюкзак, умещающий суммарный вес не больше $\overline{w}$, и при этом максимизировать суммарную цену объектов в рюкзаке. Задачу можно формализовать следующим образом:

$$\max_x \langle v, x\rangle : \qquad \langle w, x\rangle \le \overline{w}, \quad x \in \{0,1\}^n. \tag{1}$$

Здесь $w = (w_1, \dots, w_n)^T$ — вектор весов, $v = (v_1, \dots, v_n)^T$ — вектор цен, $x = (x_1, \dots, x_n)^T$ — вектор бинарных переменных x_i, которые определяют, входит ли объект i в рюкзак, или нет. Без ограничения общности можно считать, что $\max_j w_j \le \overline{w}$, $v_i > 0$, $w_i > 0$.

На практике рюкзаки вряд ли пакуют с помощью оптимизации, но задача может возникать в других контекстах. Часто она возникает как под-задача в алгоритмах решения более сложных комбинаторных проблем, например, в алгоритмах решения ЦЛП.

Объекты могут быть проектами, w_i — объём ресурсов, которые надо заложить на реализацию, v_i — прибыль с проектов. Нужно решить, за реализацию каких проектов браться, а какие отклонить, при этом уложившись в имеющийся ресурсный бюджет и максимизировав прибыль.

4.1 Линейная релаксация

Если опустить условие на бинарность переменных x_i и заменить его на выпуклое условие $x \in [0,1]^n$, то мы получим *линейную релаксацию* задачи. Она имеет следующее очевидное решение. Определим соотношения $u_i = \frac{v_i}{w_i}$ и отсортируем их по убывающей, т.е., найдём перестановку $i_1, \dots, i_n$ индексов $1, \dots, n$ такую, что $u_{i_1} \ge u_{i_2} \ge \dots \ge u_{i_n}$. Пусть далее

$$k = \arg\max_m \sum_{l=1}^{m} w_{i_l} \le \overline{w}. \tag{2}$$

Тогда положим

$$x_{i_j} = \begin{cases} 1, & j \le k, \\ \frac{\overline{w} - \sum_{l=1}^{k} w_{i_l}}{w_{i_{k+1}}}, & j = k+1, \\ 0, & j > k+1. \end{cases}$$

Иными словами, мы наполняем рюкзак приоритетно самыми ценными (по удельной цене) объектами, пока он не заполнится. Оказывается, что среди оптимальных решений линейной релаксации есть решение, в котором не более одной переменной x_i имеет дробное значение, а именно, $x_{i_{k+1}}$. Соответствующий объект называется *критическим*.

Мы получаем верхнюю и нижнюю границы на оптимальное значение v_{opt} целочисленной задачи:

$$\sum_{j=1}^{k} v_{i_j} \le v_{opt} \le \sum_{j=1}^{k} v_{i_j} + u_{i_{k+1}}(\overline{w} - \sum_{l=1}^{k} w_{i_l}) < \sum_{j=1}^{k+1} v_{i_j}. \tag{3}$$

Отметим, что зазор между значением релаксации и оптимальным значением целочисленной задачи ограничен стоимостью всего одного объекта.

Для решения линейной релаксации не обязательно полностью упорядочивать объекты по удельной стоимости. Достаточно найти критический объект, который можно найти своего рода бисекцией по множеству индексов. В основе алгоритма его поиска лежит возможность найти медиану значений u_i за линейное время. Количество вызовов алгоритма поиска медианы порядка $\log_2 n$, но каждый вызов сокращает количество потенциально критических объектов вдвое. На каждой итерации устанавливается, критическое значение удельной стоимости больше или меньше медианного значения оставшихся элементов. Алгоритм описан в работе [1].

Далее мы будем считать, что нумерация объектов произведена так, что $u_{i+1} \leq u_i$ для всех i.

4.2 Верхние и нижние границы

Линейная релаксация даёт оценки (3). Заметим, что $I = \{k+1\}$ — также допустимый рюкзак, поэтому получаем

$$\max\left(v_{k+1}, \sum_{j=1}^{k} v_j\right) \leq v_{opt} \leq \sum_{j=1}^{k} v_j + u_{k+1}(\overline{w} - \sum_{l=1}^{k} w_l).$$

Как было установлено выше, верхняя граница не больше, чем сумма выражений в максимуме, поэтому отношение оценок не превосходит 2.

Верхнюю границу можно ещё улучшить, если явным образом учесть две возможности: содержит или не содержит рюкзак критический объект. Если критический объект содержится в рюкзаке, то из числа более дорогих объектов придётся изъять объекты общим весом не менее $\sum_{i=1}^{k+1} w_i - \overline{w}$, которые будут иметь суммарную стоимость не менее $u_k \cdot \left(\sum_{i=1}^{k+1} w_i - \overline{w}\right)$. Если критический объект в рюкзак не входит, то остаточная ёмкость $\overline{w} - \sum_{i=1}^{k} w_k$ заполнится объектами с удельной стоимостью не более, чем u_{k+2}. Отсюда получаем верхнюю границу [11]

$$v_{opt} \leq \max\left(\sum_{j=1}^{k+1} v_j - u_k \cdot \left(\sum_{i=1}^{k+1} w_i - \overline{w}\right), \sum_{j=1}^{k} v_j + u_{k+2} \cdot \left(\overline{w} - \sum_{i=1}^{k} w_k\right)\right).$$

Для вычисления границ достаточно найти объекты, которые в упорядоченном по u_j списке находятся по соседству с критическим объектом, что также требует $O(n)$ операций.

4.3 Жадный алгоритм

По аналогии с описанным в предыдущем пункте решением выпуклой релаксации можно построить следующий *жадный* алгоритм.

1. упорядочиваем соотношения $u_i = \frac{v_i}{w_i}$ по убывающей
2. в цикле по $j = 1, \ldots, n$, кладём $x_{i_j} = 1$ в случае $\sum_{l=1}^{j-1} x_{i_l} w_{i_l} + w_{i_j} \leq \overline{w}$, и $x_{i_j} = 0$ иначе

Иными словами, начиная с объекта с максимальной удельной ценой, кладём объект в рюкзак, если он ещё вмещается. При этом возможны пропуски более ценных, но и более весомых объектов, если для них уже нет места.

Жадный алгоритм требует $O(n \log n)$ операций на вычисление значений u_i, их ранжировку, и проверку неравенств. Следующий пример показывает, что никаких гарантий на качество решения данная версия алгоритма не даёт.

Пример: Пусть имеются два объекта с массами $w_1 = \epsilon$, $w_2 = 1$ и ценами $v_1 = 2\epsilon$, $v_2 = 1$, и пусть $\overline{w} = 1$. Здесь ϵ — достаточно малая величина. Тогда удельные цены заданы $u_1 = 2$, $u_2 = 1$, и жадный алгоритм выдаст решение $x_1 = 1$, $x_2 = 0$ со значением функционала 2ϵ. Однако, оптимальным решением является комбинация $x_1 = 0$, $x_2 = 1$ с оптимальным значением 1.

Однако, мы можем легко улучшить результат жадного алгоритма. Пусть k задано по формуле (2). Тогда в качестве альтернативы решению, которое выдаёт жадный алгоритм, можно также положить в рюкзак предмет $k+1$ стоимостью v_{k+1}. Выбрав лучшую из альтернатив, получаем решение со значением не меньше, чем $\max\left(\sum_{j=1}^{k} v_j, v_{k+1}\right)$. С другой стороны, оптимальное значение v_{opt} ограничено сверху суммой $\sum_{j=1}^{k+1} v_j$ двух величин, входящих в максимум. Поэтому лучшая из альтернатив имеет значение функционала не меньшее, чем $\frac{v_{opt}}{2}$. Таким образом, предложенная эвристика аппроксимирует оптимальное решение с точностью до множителя не хуже, чем 2.

4.4 Аппроксимация с помощью полиномиального алгоритма

Жадный алгоритм с перебором [13]: Множитель можно произвольно улучшить, т.е., сделать сколь угодно близким к 1, при этом сохраняя полиномиальную сложность алгоритма. Пусть $X_{opt} \subset \{1, \ldots, n\}$ — оптимальное решение задачи с оптимальным значением v_{opt}, и $k \in \{1, \ldots, n\}$ — произвольное число. Пусть $i_1, \ldots, i_k \in X_{opt}$ — индексы с наибольшими стоимостями v_i среди входящих в оптимальный рюкзак. Определим $I_1 = \{i_1, \ldots, i_k\}$, $w^1 = \sum_{i \in I_1} w_i$, $v^1 = \sum_{i \in I_1} v_i$.

Построим решение $\hat{X} \subset \{1, \ldots, n\}$ следующим образом. Сначала положим в рюкзак k объектов из множества I_1, а остальные отсортируем по удельной стоимости $u_i = \frac{v_i}{w_i}$ в нисходящем порядке и, следуя философии жадного алгоритма, пройдёмся подряд по всем предметам и положим в рюкзак каждый предмет, который ещё входит. Оценим суммарную стоимость $\hat{v} = \sum_{i \in \hat{X}} v_i$ решения.

Если $X_{opt} \subset \hat{X}$, то из оптимальности X_{opt} следует, что $\hat{X} = X_{opt}$, и $\hat{v} = v_{opt}$. Допустим, что $\hat{X} \neq X_{opt}$. Пусть $j \in X_{opt}$ — индекс первого предмета из оптимального рюкзака, который не вошёл в $\hat{X}$.

Идея доказательства в том, что стоимость v_j этого объекта, с одной стороны, занимает долю не большую, чем $\frac{1}{k+1}$ в оптимальном рюкзаке, а с другой стороны, ограничивает сделанную алгоритмом ошибку (по сравнению с оптимальной стоимостью) сверху.

Определим следуюшие непересекающиеся между собой и с I_1 индексные множества.

Пусть $I_2 \subset (X_{opt} \cap \hat{X}) \setminus I_1$ — множество индексов предметов из оптимального рюкзака, которые были положены в рюкзак $\hat{X}$ до момента рассмотрения предмета j. Определим $I_3 = X_{opt} \setminus (I_1 \cup I_2)$. Пусть $I_4 \subset \hat{X} \setminus X_{opt}$ — множество индексов предметов, не входящих в оптимальный рюкзак, но положенных в $\hat{X}$ до момента рассмотрения предмета j. Пусть v^i, w^i, $i = 2, 3, 4$, определены по аналогии с v^1, w^1. Тогда справедливы следующие соотношения:

1. $v_{opt} = v^1 + v^2 + v^3$, поскольку $X_{opt} = I_1 \cup I_2 \cup I_3$
2. $\hat{v} \geq v^1 + v^2 + v^4$, поскольку $\hat{X} \supset I_1 \cup I_2 \cup I_4$
3. $w_j > \overline{w} - w^1 - w^2 - w^4$, поскольку к моменту, когда очередь подошла к предмету j, в рюкзаке осталось место для предметов с суммарным весом $\overline{w} - w^1 - w^2 - w^4$, а предмет j не уместился
4. $\overline{w} \geq w^1 + w^2 + w^3$, так как $X_{opt} = I_1 \cup I_2 \cup I_3$ — допустимая точка
5. $u_i \geq u_j$ для всех $i \in I_4$, поскольку предметы с индексом из I_4 были положены в рюкзак до рассмотрения предмета j, отсюда следует $v^4 \geq u_j w^4$
6. $u_i \leq u_j$ для всех $i \in I_3$, поскольку j — индекс первого предмета из I_3, который был рассмотрен, отсюда следует $v^3 \leq u_j w^3$
7. $v_j \leq v_i$ для всех $i \in I_1$, поскольку в I_1 вошли индексы самых дорогих предметов из оптимального рюкзака, отсюда слудует $v^1 \geq k v_j$
8. $v_j \leq v^3$, поскольку $j \in I_3$

Из них следуют соотношения:

9. из 1., 7., 8. получаем $v_{opt} \geq (k+1) v_j$
10. из 3., 4. получаем $w_j > w^3 - w^4$

11. из 5., 6., 10. получаем $v_j > v^3 - v^4$
12. из 1., 2., 11. получаем $v_{opt} - \hat{v} < v_j$
13. из 9., 12. получаем $\frac{\hat{v}}{v_{opt}} > \frac{k}{k+1}$

Для того, чтобы применить алгоритм построения решения $\hat{X}$, нужно либо знать множество I_1, либо быть уверенным в том, что оптимальный рюкзак X_{opt} содержит не более, чем k объектов. И то, и другое можно проверить, просто перебрав в качестве множества I_1 все подмножества мощности не более k, и применив для каждого такого подмножества жадный алгоритм к оставшимся объектам. Это приводит к $O(n^k)$ случаям, для каждого из которых нужно потратить $O(n)$ операций на построение $\hat{X}$. К этому прибавляются $O(n \log n)$ операций для сортировки объектов по удельным стоимостям. В итоге для фиксированного k получаем полиномиальный алгоритм.

4.5 Решение динамическим программированием

Рассмотрим решение задачи о рюкзаке и его значение функционала как функцию от параметра $\overline{w}$. Функция $v_{opt}(\overline{w})$ — кусочно-постоянная (так как есть только конечное количество значений, которых может принять v_{opt}) и монотонно возрастающая. Более того, $v_{opt}(\overline{w}) = -\infty$ для $\overline{w} < 0$, $v_{opt}(\overline{w}) = 0$ для $0 \le \overline{w} < \min_i w_i$ и $v_{opt}(\overline{w}) = \sum_{i=1}^n v_i$ для $\overline{w} \ge \sum_{i=1}^n w_i$. Поэтому для описания функции нужно хранить конечное количество информации, а именно, оптимальные рюкзаки, их значение функционала и соотвествующий интервал значений параметра $\overline{w}$. Знание функции очевидно позволяет найти оптимальное решение исходной задачи.

Для эффективного вычисления функции учтём ещё явным образом данные задачи $(v_1, w_1), \ldots, (v_n, w_n)$ в аргументе. Тогда можно выписать следующую рекурсию:

$$v_{opt}(\overline{w}; (v_1, w_1), \ldots, (v_k, w_k)) = \max\left(v_{opt}(\overline{w}; (v_1, w_1), \ldots, (v_{k-1}, w_{k-1})), v_k + v_{opt}(\overline{w} - w_k; (v_1, w_1), \ldots, (v_{k-1}, w_{k-1}))\right).$$

При этом в случае, когда максимум достигается на первом выражении, соответствующий рюкзак для k объектов не отличается от рюкзака для $k-1$ объектов, в противном случае к нему добавляется предмет k.

Начиная с функции

$$v_{opt}(\overline{w}; \emptyset) = \begin{cases} -\infty, & \overline{w} < 0, \\ 0, & \overline{w} \ge 0 \end{cases}$$

можно последовательно построить функции $v_{opt}(w; (v_1, w_1))$, $v_{opt}(w; (v_1, w_1), (v_2, w_2))$ вплоть до функции $v_{opt}(w; (v_1, w_1), \ldots, (v_n, w_n))$. При этом целесообразно заранее отсортировать объекты по порядку убывающей удельной стоимости $u_i = \frac{v_i}{w_i}$.

Так как при вычислении значения функции $v_{opt}(w; (v_1, w_1), \ldots, (v_k, w_k))$ используются только значения функции $v_{opt}(w'; (v_1, w_1), \ldots, (v_{k-1}, w_{k-1}))$ для $w' \le w$, а для решения исходной задачи нужно вычислить значение при $w = \overline{w}$, то и функции более низких уровней достаточно посчитать на интервале $[0, \overline{w}]$.

Если все веса являются натуральными числами, то в этом интервале достаточно посчитать функцию $v_{opt}(w)$ только для целых w, которых $\overline{w}+1$ штук. Поэтому можно обойтись вычислением максимум $O(n\overline{w})$ значений. Такой алгоритм называют *псевдо-полиномиальным*.

Алгоритм можно реализовать следующим образом. Кусочно-постоянную функцию $v_{opt}(w; (v_1, w_1), \ldots, (v_k, w_k))$ можно представить упорядоченным списком из m_k точек $(v_{k,1}, w_{k,1}), \ldots, (v_{k,m_k}, w_{k,m_k})$ разрыва графа функции, таким образом, что функция выражается в виде

$$v_{opt}(w; (v_1, w_1), \ldots, (v_k, w_k)) = v_{k,j}, \qquad j = \arg\max_l w_{k,l} : \quad w_{k,l} \le w.$$

Здесь обе последовательности $v_{k,l}, w_{k,l}$ строго монотонно возрастают по l для любого $k = 0, \ldots, n$.

Начиная с $m_0 = 1$, $(v_{0,1}, w_{0,1}) = (0, 0)$, последовательность $\{(v_{k,l}, w_{k,l})\}_{l=1,\ldots,m_k}$ строится на основе $\{(v_{k-1,l}, w_{k-1,l})\}_{l=1,\ldots,m_{k-1}}$ следующим образом. Объединение двух последовательностей

$$\{(v_{k-1,l}, w_{k-1,l})\}_{l=1,\ldots,m_{k-1}}, \quad \{(v_k + v_{k-1,l}, w_k + w_{k-1,l})\}_{l=1,\ldots,m_{k-1}}$$

упорядочивается по второму аргументу $w_{k,l}$ по возрастающей, а для точек с равными вторыми аргументами по первому аргументу по убывающей. Потом удаляются все пары $(v_{k,l}, w_{k,l})$ такие, что $v_{k,l} \le v_{k,l-1}$ или $w_l > \overline{w}$.

С каждой парой $(v_{k,l}, w_{k,l})$ ассоциирован рюкзак $X_{k,l} \subset \{1, \ldots, n\}$, который также строится рекурсивно. Начиная с $X_{0,1} = \emptyset$, полагаем $X_{k,l} = X_{k-1,j}$ если $(v_{k,l}, w_{k,l}) = (v_{k-1,j}, w_{k-1,j})$ и $X_{k,l} = X_{k-1,j} \cup \{k\}$ если $(v_{k,l}, w_{k,l}) = (v_k + v_{k-1,j}, w_k + w_{k-1,j})$ для некоторого j.

Оптимальный рюкзак в итоге получается в виде индексного подмножества X_{n,m_n}.

В описанном выше алгоритме объекты добавлялись по отдельности, и появление каждого нового объекта сопровождалось обновлением функции $v_{opt}(w)$. Алгоритм можно модифицировать таким образом, что объекты сначала собираются в дизъюнктные подмножества, для каждого из которых строится подобная функция, а потом группы объединяются [5]. Если на итерации метода, представленного выше, строились попарные суммы элементов множеств

$$\{(v_{k-1,l}, w_{k-1,l})\}_{l=1,\ldots,m_{k-1}}, \qquad \{(0,0), (v_k, w_k)\},$$

то при объединении двух подмножеств R, S элементов строятся попарные суммы элементов множеств

$$\{(v_{R,l}, w_{R,l})\}_{l=1,\ldots,m_R}, \qquad \{(v_{S,l}, w_{S,l})\}_{l=1,\ldots,m_S},$$

соответствующих подмножествам R, S.

Возможно также многоуровневое объединение всё более укрупняющихся подмножеств, пока не построится функция для множества всех объектов.

Аналогично динамическим программированием можно построить рюкзак наименьшего веса, имеющий определённую цену.

4.6 Гибридный алгоритм: жадный и динамическое программирование

Следующий алгоритм с полиномиальной сложностью, решающий задачу с относительной точностью $(1-\epsilon)^{-1}$, был предложен Ибаррой и Кимом [7].

Пусть $\epsilon > 0$ задано. Пусть далее $v_{opt} \leq \overline{v} \leq 2v_{opt}$ — верхняя граница на цену v_{opt} оптимального рюкзака, например, полученная решением линейной релаксации. Поделим множество объектов на *дорогие* и *дешёвые*, в зависимости от того, превосходит или нет их стоимость порог $\frac{\epsilon}{3}\overline{v}$. Определим также $\delta = (\frac{\epsilon}{3})^2\overline{v}$, $q = \lfloor (\frac{3}{\epsilon})^2 \rfloor$. Для дорогих объектов отшкалируем и округлим их стоимость, присвоив виртуальную стоимость $\xi_j = \lfloor \frac{v_j}{\delta} \rfloor$. Таким образом,

$$\delta \cdot \xi_j \leq v_j < \delta \cdot (\xi_j + 1).$$

Так как любой допустимый рюкзак имеет суммарную стоимость, не превышающую v_{opt}, стоимость дорогих объектов в нём также не превосходит $v_{opt} \leq \overline{v}$. Но тогда виртуальная стоимость этих объектов не может превосходить $\lfloor \frac{\overline{v}}{\delta} \rfloor = q$. С другой стороны, в любом допустимом рюкзаке не может находиться более $\frac{3}{\epsilon} \cdot \frac{v_{opt}}{\overline{v}}$ дорогих объектов, так как их суммарная стоимость не может превысить v_{opt}.

Алгоритм состоит из четырёх фаз:

- упорядочивание объектов по удельной стоимости u_i
- фаза динамического программирования
- фаза жадного алгоритма
- выбор лучшего из найденных решений

Фаза динамического программирования работает только с дорогими объектами. Для каждого заданного значения $\xi \in \{0, \ldots, q\}$ она определяет рюкзак наименьшего веса с виртуальной стоимостью ξ. Обозначим множество объектов, входящих в этот рюкзак, через $\tilde{I}_\xi^{exp}$. Заметим, что для данного ξ может и не существовать рюкзака из дорогих объектов с виртуальной стоимостью ξ. Так как значений, для которых нужно вычислить этот рюкзак, имеется не более, чем $q+1$, а объектов имеется n, фаза динамического программирования требует $O(nq)$ операций.

Фаза жадного алгоритма добавляет в каждый из не более, чем $q+1$ рюкзаков $\tilde{I}_\xi^{exp}$ с дорогими объектами ещё дешёвые объекты, причём в порядке убывания их удельной стоимости. Если текущий дешёвый объект не умещается в рюкзаке, алгоритм переходит к следующему. Обозначим множество дешёвых объектов,

которых алгоритм добавляет, через $\tilde{I}_\xi^{chp}$, а суммарный рюкзак через $\tilde{I}_\xi = \tilde{I}_\xi^{exp} \cup \tilde{I}_\xi^{chp}$. Эта фаза также требует не более, чем $O(nq)$ операций.

В конце из не более, чем $q+1$ полученных рюкзаков выбирается рюкзак максимальной стоимости.

В итоге это приводит к сложности $O(n\epsilon^{-2}) + O(n \log n)$, вместе с начальным упорядочиванием по u_j.

Проанализируем качество найденного алгоритмом рюкзака. Обозначим его через $\hat{I} \subset \{1, \ldots, n\}$, со стоимостью $\hat{v}$. Пусть оптимальный рюкзак I_{opt} имеет стоимость v_{opt}. Поделим I_{opt} на подмножества I_{opt}^{exp}, I_{opt}^{chp} с дорогими и дешёвыми объектами, соответственно, имеющие суммарные стоимости v_{opt}^{exp}, v_{opt}^{chp}. Пусть виртуальная стоимость множества I_{opt}^{exp} равна $\hat{\xi}$. Рассмотрим рюкзак $\tilde{I}_{\hat{\xi}}$ со стоимостью $\tilde{v}$. Он распадается на подмножества $\tilde{I}_{\hat{\xi}}^{exp}$, $\tilde{I}_{\hat{\xi}}^{chp}$ с дорогими и дешёвыми объектами, имеющие суммарные стоимости $\tilde{v}^{exp}$, $\tilde{v}^{chp}$.

Тогда имеем неравенства

$$\hat{v} \geq \tilde{v} = \tilde{v}^{exp} + \tilde{v}^{chp}, \qquad \delta \cdot \hat{\xi} = \delta \cdot \sum_{j \in \tilde{I}_{\hat{\xi}}^{exp}} \xi_j \leq \sum_{j \in \tilde{I}_{\hat{\xi}}^{exp}} \tilde{v}_j = \tilde{v}^{exp},$$

$$v_{opt}^{exp} = \sum_{j \in I_{opt}^{exp}} v_j < \delta \cdot \sum_{j \in I_{opt}^{exp}} (\xi_j + 1) = \delta \cdot \hat{\xi} + \delta \cdot |I_{opt}^{exp}| \leq \delta \cdot \hat{\xi} + (\frac{\epsilon}{3})^2 \overline{v} \cdot \frac{3}{\epsilon} \cdot \frac{v_{opt}}{\overline{v}} = \delta \cdot \hat{\xi} + \frac{\epsilon}{3} v_{opt}.$$

Более того, так как рюкзак $\tilde{I}_{\hat{\xi}}^{exp}$ имеет минимальный вес среди рюкзаков с виртуальной стоимостью $\hat{\xi}$, имеем

$$\sum_{j \in \tilde{I}_{\hat{\xi}}^{exp}} w_j \leq \sum_{j \in I_{opt}^{exp}} w_j.$$

Поэтому при жадном выборе из множества дешёвых объектов, приведшем к рюкзаку $\tilde{I}_{\hat{\xi}}^{chp}$, имелась в наличии остаточная ёмкость $\overline{w} - \sum_{j \in \tilde{I}_{\hat{\xi}}^{exp}} w_j$ не меньшая, чем для оптимального выбора объектов из этого множества, приведшего к рюкзаку I_{opt}^{chp}. Даже при одинаковых остаточных ёмкостях оптимальный рюкзак не может быть дороже жадного на более, чем стоимость критического объекта. Но так как критический объект здесь из числа дешёвых, получаем

$$v_{opt}^{chp} - \tilde{v}^{chp} \leq \frac{\epsilon}{3} \overline{v} \leq \frac{2\epsilon}{3} v_{opt}.$$

Отсюда получаем

$$v_{opt} - \hat{v} \leq v_{opt}^{exp} + v_{opt}^{chp} - \tilde{v}^{exp} - \tilde{v}^{chp} < \frac{\epsilon}{3} v_{opt} + \frac{2\epsilon}{3} v_{opt} = \epsilon v_{opt}, \qquad \frac{\hat{v}}{v_{opt}} \geq 1 - \epsilon.$$

4.7 Алгоритм ветвей и границ

В этом разделе мы ознакомимся с алгоритмом решения задачи о рюкзаке типа ветвей и границ. Такие алгоритмы сегодня являются стандартным методом решения смешанно-целочисленных задач с выпуклыми релаксациями. В задаче о рюкзаке можно построить специализированный алгоритм, учитывающий особенную форму решения линейной релаксации.

Как было сказано в п. 4.1, в решении линейной релаксации только критический объект $k+1$ может входить в оптимальный (для релаксации) рюкзак в дробном объёме. Если в решении релаксации переменная x_{k+1} равна нулю, т.е., предыдущие k объектов насытили ограничение на суммарный вес, то линейная релаксация нашла оптимальное решение исходной задачи. Допустим, что $x_{k+1} \in (0, 1)$. Тогда решение релаксации не допустимо для исходной задачи, так как в последней в отношении критического объекта есть только два варианта: либо он входит в оптимальный рюкзак, либо нет. Соответственно, можно для каждого из этих вариантов рассмотреть новую задачу о рюкзаке, в которой критический объект изъят из списка потенциально входящих в рюкзак объектов.

Если $x_{k+1} = 0$, то в новой задаче ничего более не меняется. Если $x_{k+1} = 1$, то мы рассматриваем новую задачу с параметром $\overline{w} \leftarrow \overline{w} - w_{k+1}$, изымаем из списка объектов все объекты, вес которых превосходит новое значение $\overline{w}$, а к функционалу цены прибавляем константу v_{k+1}. Таким образом, вместо исходной

задачи создаются две т.н. *дочерние* задачи, в которых критический объект отсутствует. Эту процедуру называют *ветвлением* по этому объекту.

Для каждой дочерней задачи снова можно решить линейную релаксацию и поветвить по соответствующему критическому объекту в случае дробного значения соответствующей переменной в решении релаксации. Таким образом алгоритм постепенно строит бинарное дерево. Задачи, строящиеся алгоритмам, соответствуют *узлам* дерева. Так как при каждом ветвлении из списка изымается как минимум один объект, в узле, лежащем на глубине m в дереве, не может быть более $n - m$ объектов в списке задачи. Поэтому количество потенциальных узлов в дереве ограничено, и алгоритм останавливается за конечное время.

Обход дерева может осуществляется алгоритмами общего характера.

Рассмотрим детали алгоритма при решении линейной релаксации в дочерних узлах. Если дочерний узел характеризуется присваиванием критической переменной значения $x_{k+1} = 0$, то к k объектам, уже лежащем в рюкзаке в решении релаксации родительского узла, добавляются дополнительные объекты по порядку убывающей удельной цены, пока не будет найден новый критический объект. Этот шаг называется *движением вперёд*. Если же в дочернем узле $x_{k+1} = 1$, то из рюкзака следует удалять объекты по порядку возрастающей удельной цены, пока не будет найден новый критический объект. Этот шаг называется *движением назад*. Таким образом, решение релаксации в родительском узле можно использовать как начальную точку для решения релаксаций в дочерних узлах. Этот принцип общий для всех алгоритмов типа ветвей и границ.

По ходу работы алгоритма значение наилучшего доселе найденного рюкзака монотонно растёт. Оно является нижней границей для оптимального значения задачи. С другой стороны, можно построить глобальную верхнюю границу, которая монотонно падает. Эта граница считается как максимум верхних границ по всем узлам, для которых эта граница не посчитана в обоих дочерних узлах. Заметим, что в дочерних узлах верхняя граница не может быть выше, чем в родительском узле. Как только узел, в котором достигается максимум верхней границы, разветвится и посчитаются верхние границы в его дочерних узлах, глобальная верхняя граница может понизиться.

Таким образом, нижняя и верхняя граница на оптимальное значение движутся навстречу друг к другу, и *зазор* между ними уменьшается. Так как этот зазор ограничивает ошибку по значению функционала, которая будет сделана при выборе достигнутого наилучшего решения, можно остановить алгоритм при достижении желаемой точности.

4.8 Фиксация переменных и упрощение задачи

Сравнением нижних и верхних границ иногда удаётся установить, что тот или иной объект обязан присутствовать, или наоборот, отсутствовать, в оптимальном рюкзаке.

Допустим, что в наличии уже есть допустимое решение со значением $\underline{v}$. Чтобы доказать, что некоторый объект k обязательно должен присутствовать в оптимальном рюкзаке, достаточно построить верхнюю границу $\overline{v}_0^k < \underline{v}$ для некой вспомогательной задачи, в которой переменной x_k присвоено значение 0, т.е., из которой объект k изъят. Это означает, что без объекта k даже невозможно достичь значения уже найденного решения. Так как при наличии уже найденного решения мы заинтересованы в нахождении рюкзака со строго большим значением, то знак в неравенстве можно считать не строгим. Неравенство $\overline{v}_0^k \leq \underline{v}$ означает, что без объекта k невозможно построить строго лучшее решение, чем уже найденное.

С другой стороны, чтобы доказать, что некоторый объект k обязательно должен отсутствовать в оптимальном рюкзаке, достаточно построить верхнюю границу $\overline{v}_1^k < \underline{v}$ для вспомогательной задачи, в которой переменной x_k присвоено значение 1, или эквивалентно, из которой объект k изъят, ёмкость рюкзака $\overline{w}$ заменена на $\overline{w} - w_k$, а к функционалу цены прибавлена константа v_k.

На обязательное присутствие в рюкзаке проверяются объкеты с удельной стоимостью больше критической, а на обязательное отсутствие — с меньшей. Пусть J_1 — множество индексов объектов, которые обязательно должны присутствовать в оптимальном рюкзаке, а J_0 — множество индексов объектов, которые обязательно должны в нём отсутствовать. Тогда вместо исходной задачи со списком объектов I и ёмкостью $\overline{w}$ можно решать упрощённую задачу со списком объектов $I \setminus (J_0 \cup J_1 \cup J_H)$ и ёмкостью $\overline{w}' = \overline{w} - \sum_{k \in J_1} w_k$, где J_H — список объектов с весом, превосходящим $\overline{w}'$. В конце в найденный рюкзак надо добавить объекты из J_1 и прибавить сумму $\sum_{k \in J_1} v_k$ к его цене.

Этот процесс редукции задачи можно повторять многократно, пока она не перестанет упрощаться. Прототип этого подхода был представлен в работе [9].

4.9 Ядро и упрощённая задача

Пусть индексы $i \in I$ объектов упорядочены по удельной стоимости по убывающей, и $I_{opt} \subset I$ — множество индексов объектов в оптимальном рюкзаке. Определим множества индексов

$$J_1 = \{1, \ldots, j_1 - 1\}, \qquad j_1 = \min\{j \notin I_{opt}\},$$

$$J_0 = \{j_0 + 1, \ldots, n\}, \qquad j_0 = \max\{j \in I_{opt}\}$$

и упрощённую задачу с множеством $I \setminus (J_0 \cup J_1)$ и ёмкостью $\overline{w} - \sum_{i \in J_1} w_i$. Если индексы j_0, j_1 заранее известны, то для нахождения оптимального рюкзака достаточно решить упрощённую задачу, а в её оптимальный рюкзак добавить объекты из множества J_1. Подмножество $I \setminus (J_0 \cup J_1)$ называется *ядром*.

В реальности ядро заранее не известно. Однако, во многих случаях его размер составляет только малую долю от общего количества объектов. Ясно, что если ядро пустое, то оптимальный рюкзак содержит все объекты вплоть до критического исключительно. Если ядро непустое, то оно содержит критический объект.

Можно реализовать следующую эвристику. Фиксируем n_0 и определяем множество

$$J_C = \{\max(1, k + 1 - n_0), \ldots, \min(k + 1 + n_0, n)\}$$

мощностью не более $2n_0 + 1$, где $k + 1$ — индекс критического объекта. Далее решаем задачу с множеством объектов J_C и ёмкостью $\overline{w} - \sum_{j < k+1-n_0} w_j$. В полученный рюкзак добавляем объекты $1, \ldots, \max(1, k + 1 - n_0)$. Обратим внимание на то, что для определения J_C не обязательно упорядочивать всё множество индексов по удельной стоимости. Достаточно найти критический объект и по n_0 объектов, ближайших к нему справа и слева. Это можно сделать за $O(n)$ операций. Далее достаточно упорядочить J_C. Прототип такого алгоритма был представлен в работе [1].

Если J_C содержит ядро, то найденное решение будет оптимальным. Иначе получаем субоптимальное решение. Если решать задачу с множеством объектов J_C динамическим программированием, то можно добавлять объекты в J_C и дорешивать производные задачи с меньшим количеством операций, чем если бы они решались с нуля.

4.10 Задача о сумме подмножеств

Представленные до сих пор алгоритмы в той или иной степени использовали ранжировку объектов по удельной стоимости. Поэтому следует ожидать, что задачи, в которых удельная стоимость постоянна, т.е., $v_i = w_i$, будут особо сложными. Как частный случай можно выделить следующую *задачу о сумме подмножеств*:

Задача 1: Пусть даны веса $w_i \in \mathbb{N}_+$ объектов, $i = 1, \ldots, n$, и ёмкость $\overline{w} \in \mathbb{N}_+$. Найти подмножество $I \subset \{1, \ldots, n\}$ такое, что $\sum_{i \in I} w_i = \overline{w}$.

Иными словами, имеет ли оптимальный рюкзак максимально возможный суммарный вес $\overline{w}$? Эта задача НП-полная, и одно время даже использовалась в крипто-системах (Меркль-Хеллман) [12].

В этой системе словом, которое нужно зашифровать, является подмножество I, в зашифрованном виде представленное суммой $\overline{w} = \sum_{i \in I} w_i$. Для расшифровки стороннему лицу нужно решить инстанцию задачи. Пользователь, обладающий ключом, имеет возможность переформулировать задачу в эквивалентной, решаемой за линейное время форме. В этой форме последовательность весов w'_i *супервозрастающая*, т.е., удовлетворяет условию

$$w'_{i+1} > \sum_{j=1}^{i} w'_j.$$

Ясно, что для решения задачи нужно упорядочить веса в убывающем (обратном) порядке и применить жадный алгоритм. Веса w_i и w'_i связаны условием $w_i = r w'_i(q)$, где r, q — взаимно простые числа, а

супервозрастающие вычеты w'_i по модулю q являются частью ключа. Для расшифровки нужно построить модифицированную ёмкость $\overline{w}' = \overline{w} \cdot r^{-1}(q)$ и решить задачу в немного изменённом виде:

Задача 2: Пусть даны числа $w'_1, \ldots, w'_n, \overline{w}' \in \{1, \ldots, q-1\}$. Найти подмножество $I \subset \{1, \ldots, n\}$ такое, что $\sum_{i \in I} w'_i = \overline{w}'(q)$?

То обстоятельство, что равенство ищется по модулю q, не сильно усложняет задачу. Так как $\sum_{i=1}^{n} w'_i < nq$, достаточно решить не более n классических задач о сумме подмножеств, чтобы найти ответ в модифицированной задаче.

Проблема системы Меркла-Хеллмана состоит в том, что пара q, r, сопоставляющая последовательности $\{w_i\}$ некую супервозрастающую последовательность, не единственна. Более того, такую пару можно построить за полиномиальное время, и, таким образом, найти (возможно отличающийся от предусмотренного) ключ к расшифровке [15].

В модифицированной задаче о сумме подмножеств наблюдается *фазовый переход*. Пусть n весов w_i и ёмкость $\overline{w}$ выбраны случайно и равномерно из множества $\{1, \ldots, q-1\}$. Тогда самые сложные задачи находятся в области, в которой $q \sim 2^n$ [8]. Соотношение

$$d = \frac{n}{\log_2 \max_i w_i}$$

называется *плотностью* задачи. Самые сложные задачи, т.о., имеют плотность около $d = 1$. Для задач с меньшей [4] и большей [10] плотностью имеются эвристические алгоритмы, решающие их довольно успешно.

Для задач с плотностью $d \approx 1$ алгоритмы, описанные в литературе, не намного проще полного перебора.

Алгоритм Хоровица-Сани: Базовый алгоритм имеет сложность $O(2^{n/2})$ и считает все возможные суммы, составленные из весов из двух подмножеств размера $\frac{n}{2}$ [5]. Алгоритм для решения задачи 1:

- разбить множество $\{1, \ldots, n\}$ на два подмножества I_1, I_2 по $\approx \frac{n}{2}$ элементов
- посчитать все суммы вида $\sum_{i \in I \subset I_1} w_i$, $\sum_{i \in I \subset I_2} w_i$
- упорядочить первое множество сумм s^1_k по возрастанию в список S_1
- упорядочить второе множество сумм s^2_k по убыванию в список S_2
- начиная с $k_1 = k_2 = 1$, сравнивать сумму $s = s^1_{k_1} + s^2_{k_2}$ с $\overline{w}$:
- если $s < S$, $k_1 \leftarrow k_1 + 1$
- если $s > S$, $k_2 \leftarrow k_2 + 1$
- если $s = S$, решение найдено

Алгоритм Шрёппеля-Шамира [14]: Алгоритм Хоровица-Сани требует $O(2^{n/2})$ памяти. Алгоритм Шрёппеля-Шамира является его модификацией, в которой это значение можно понизить до $O(2^{n/4})$ разбиением множества весов на 4 подмножества, из которых формируются 4 списка U_1, U_2, U_3, U_4 сумм u^j_k, $j = 1, 2, 3, 4$. Списки U_1, U_3 упорядочиваются, U_1 по возрастанию, U_3 по убыванию. Значения сумм из списков S_1, S_2 в алгоритме Хоровица-Сани далее генерируются динамично.

Поясним схему генерации. Пусть дан порог α, и мы хотим найти наибольшую сумму вида $u^1_k + u^2_l$ из списка S_1, не превосходящую α. Для каждого фиксированного $l = 1, \ldots, 2^{n/4}$ бинарным поиском можно найти сумму с таким свойством среди подмножества сумм вида $u^1_k + u^2_l$. Это задаёт нам некоторую функцию $k = k(l)$. Наибольшую сумму из списка S_1 далее можно искать уже среди сумм вида $u^1_{k(l)} + u^2_l$.

Допустим теперь, что мы хотим найти следующую в возрастающем порядке сумму вида $u^1_k + u^2_l$ из списка S_1. Ясно, что её надо искать среди сумм вида $u^1_{k(l)+1} + u^2_l$. Таким образом, чтобы пройтись по упорядоченному списку сумм из S_1, нужно хранить и обновлять только $2^{n/4}$ индексов $k(l)$.

В итоге получаем следующую схему. Суммы s^1_m из списка S_1 имеют вид $u^1_k + u^2_l$, а суммы s^2_m из списка S_2 имеют вид $u^3_k + u^4_l$. При генерации s^1_m, для каждого l в памяти держится только одно актуальное значение $k(l)$ и соответствующая сумма $u^1_{k(l)} + u^2_l$. Из них выбирается подходящее значение $s^1_m = u^1_{k(l^*)} + u^2_{l^*}$, после

чего соответствующее значение $k(l^*)$ увеличивается на 1. Аналогично формируются суммы s_m^2 из списка S_2. Алгоритм Шрёппеля-Шамира всё ещё имеет сложность $O(2^{n/2})$ по времени.

Дальнейшими вариациями можно улучшить и количество операций до $O(2^{n/3})$ [6] и быстрее [2],[3].

Список литературы

[1] E. Balas and E. Zemel. An algorithm for large zero-one knapsack problems. *Operations Research*, 28:1130–1154, 1980.

[2] Anja Becker and Antoine Joux. *Improved generic algorithms for hard knapsacks*, volume 6632 of *LNCS*, pages 364–385. Springer, 2011.

[3] Xavier Bonnetain, Rémi Bricout, André Schrottenloher, and Yixin Shen. Improved classical and quantum algorithms for subset-sum. In Shiho Moriai and Huaxiong Wang, editors, *Advances in Cryptology - ASIACRYPT 2020*, volume 12492 of *Lecture Notes in Computer Science*, pages 633–666, Berlin, Heidelberg, 2020. Springer.

[4] M.J. Coster and A. Joux et al. Improved low-density subset sum algorithms. *Computational Complexity*, 2:111–128, 1992.

[5] E. Horowitz and S. Sahni. Computing partitions with applications to the knapsack problem. *J. Assoc. Comp. Mach.*, 21(2):277–292, 1974.

[6] N. Howgrave-Graham and A. Joux. *New generic algorithms for hard knapsacks*, volume 6110 of *LNCS*, pages 235–256. Springer, 2010.

[7] Oscar H. Ibarra and Chul E. Kim. Fast approximation algorithms for the knapsack and sum of subset problems. *J. Assoc. Comput. Machinery*, 22(4):463–468, 1975.

[8] Russell Impagliazzo and Moni Naor. Efficient cryptographic schemes provably as secure as subset sum. *J. Cryptology*, 9:199–216, 1996.

[9] G.P. Ingargiola and J.F. Korsh. A reduction algorithm for zero-one single knapsack problems. *Management Science*, 20:460–463, 1973.

[10] A. Joux and L. Granboulan. *A practical attack against knapsack based hash functions*, volume 950 of *LNCS*, pages 58–66. Springer, Heidelberg, 1994.

[11] S. Martello and P. Toth. An upper bound for the zero-one knapsack problem and a branch-and-bound algorithm. *Europ. J. Oper. Res.*, 1:169–175, 1977.

[12] Ralph Merkle and Martin Hellman. Hiding information and signatures in trapdoor knapsacks. *IEEE Trans. Information Theory*, 24(5):525–530, 1978.

[13] S. Sahni. Approximate algorithms for the 0-1 knapsack problem. *Journal of ACM*, 22:115–124, 1975.

[14] R. Schroeppel and A. Shamir. A $T = O(2^{n/2})$, $S = O(2^{n/4})$ algorithm for certain NP-complete problems. *SIAM J. Comput.*, 10(3):456–464, 1981.

[15] A. Shamir. A polynomial time algorithm for breaking the basic Merkle-Hellman cryptosystem. *IEEE Trans. Information Theory*, 30(5):699–704, 1984.